



Natural Language Processing Applications



BITS Pilani

Pilani Campus

Dr. Chetana Gavankar, Ph.D,

IIT Bombay-Monash University Australia

Chetana.gavankar@pilani.bits-pilani.ac.in



BITS Pilani
Pilani Campus

Session 2:

These slides are prepared by the instructor, with grateful acknowledgement of Prof. Dan Jurafsky and many others who made their course materials freely available online.

Agenda



01

Introduction & Overview

What are spell/grammar checkers?

02

Dictionary-Based Techniques

Edit distance, trie structures

03

Statistical & Probabilistic Methods

n-grams, noisy channel model

04

Machine Learning Approaches

Neural networks, transformers

05

Grammar Checking Techniques

POS tagging, parsing, rules

06

Evaluation & Challenges

Metrics, datasets, limitations

Spell checkers

innovate

achieve

lead

What Are They?

- Spell checkers detect and correct misspelled words in text
- Grammar checkers identify grammatical errors and suggest corrections
- Both rely on NLP, linguistics, and ML techniques
- Applications: word processors, IDEs, email clients, search engines

Error Categories

Non-Word Errors

eg. 'recieve' → 'receive'

Real-Word Errors

eg. 'their' vs 'there'

Syntactic Errors

eg. 'She go to school'

Semantic Errors

eg. 'colorless green ideas'

Punctuation Errors

eg. missing commas, apostrophes

Dictionary-Based Techniques



Edit Distance (Levenshtein)

Measures minimum number of single-character edits (insertions, deletions, substitutions) to transform one word into another. Used to find the closest dictionary match.

► *receiv → receive: distance = 1*

Trie / Prefix Trees

Tree-based data structure for efficient dictionary lookup. Each node represents a character. Supports $O(k)$ lookup where k = word length.

► *Fast autocomplete & spell suggestion*

Soundex / Phonetic Algorithms

Encodes words based on pronunciation to find phonetically similar words. Useful for homophones and phonetic misspellings.

► *Smith ↔ Smythe → same Soundex code*

BK-Trees

Metric tree structure optimized for approximate string matching using edit distance. Enables sub-linear search for similar words.

► *Efficiently finds all words within k edits*

Quick intuition guide



① Edit Distance



Precise typo correction

Finds the true minimum edits between two words. Best accuracy for short-distance fixes.

Complexity: $O(m \cdot n)$

② Trie



Fast lookup & autocomplete

Instant prefix queries. Perfect for search boxes, autocomplete dropdowns.

Complexity: $O(k)$

③ Soundex



Pronunciation-based errors

Catches phonetic typos like 'fone → phone'. Great for voice input & OCR.

Complexity: $O(n)$

④ BK-Tree



Scalable fuzzy search

Efficient search over large dictionaries with edit distance tolerance.

Complexity: $O(\log n)$

Statistical & Probabilistic Methods



Noisy Channel Model

$$\operatorname{argmax} P(w|e) = \operatorname{argmax} P(e|w) \times P(w)$$

$P(w)$ — Language model: prior probability of word w

$P(e|w)$ — Error model: probability of error e given intended word w

Find the most probable intended word

N-gram Language Models

Estimate word probability based on previous $n-1$ words. Bigrams, trigrams capture local context for real-word error detection.

Bayesian Correction

Uses prior probability + likelihood. Combines corpus frequency with error patterns learned from training data to rank suggestions.

Key Insight:

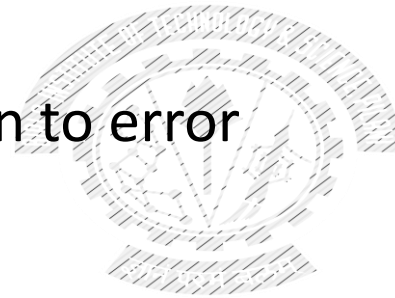
Statistical methods work WITHOUT exhaustive rules — they learn patterns from large text corpora, making them adaptable across domains and languages.

Common corpora: Google Books, Wikipedia, CommonCrawl, Brown Corpus



Spellcheck Candidate generation

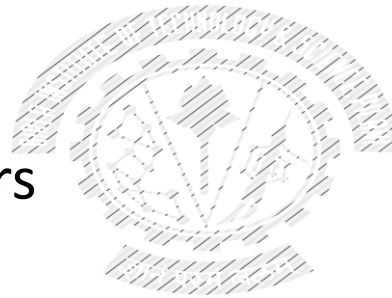
- Words with similar spelling
 - Small edit distance to error
- Words with similar pronunciation
 - Small edit distance of pronunciation to error



Damerau-Levenshtein edit distance



- Minimal edit distance between two strings, where edits are:
 - Insertion
 - Deletion
 - Substitution
 - Transposition of two adjacent letters



Words within 1 of across

Error	Candidate Correction	Correct Letter	Error Letter	Type
acress	actress	t	-	deletion
acress	cross	-	a	insertion
acress	caress	ca	ac	transposition
acress	access	c	r	substitution
acress	across	o	e	substitution
acress	acres	-	s	insertion
acress	acres	-	s	insertion

Candidate generation

- 80% of errors are within edit distance 1
- Almost all errors within edit distance 2
- Also allow insertion of space or hyphen
 - thisidea -> this idea
 - inlaw -> in-law



How do you generate the candidates?

1. Run through dictionary, check edit distance with each word
2. Generate all words within edit distance $\leq k$ (e.g., $k = 1$ or 2) and then intersect them with dictionary
3. Use a character k -gram index and find dictionary words that share “most” k -grams with word (e.g., by Jaccard coefficient)
4. Compute them fast with a Levenshtein finite state transducer
5. Have a precomputed map of words to possible corrections

What is $P(W)$?

- We see an observation x of a misspelled word
- Find the correct word $\hat{w}$

$$\hat{w} = \operatorname{argmax}_{w \in V} P(w | x)$$

$$= \operatorname{argmax}_{w \in V} \frac{P(x | w)P(w)}{P(x)}$$

$$= \operatorname{argmax}_{w \in V} P(x | w)P(w)$$

What's $P(w)$?



Unigram Prior probability

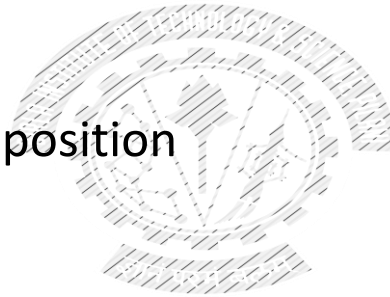
- Counts from 404,253,213 words in Corpus of Contemporary English (COCA)

word	Frequency of word	P(word)
actress	9,321	.0000230573
cress	220	.0000005442
caress	686	.0000016969
access	37,038	.0000916207
across	120,844	.0002989314
acres	12,874	.0000318463



Channel model probability

- Misspelled word $x = x_1, x_2, x_3 \dots x_m$
- Correct word $w = w_1, w_2, w_3, \dots, w_n$
- $P(x|w)$ = probability of the edit
 - (deletion/insertion/substitution/transposition)



Computing error probability: confusion matrix



```
del[x,y]:      count(xy typed as x)
ins[x,y]:      count(x typed as xy)
sub[x,y]:      count(x typed as y)
trans[x,y]:    count(xy typed as yx)
```

Insertion and deletion conditioned on previous character

Generating the confusion matrix

- [Peter Norvig's list of errors](#)
- [Peter Norvig's list of counts of single-edit errors](#)
- All Peter Norvig's ngrams data links: <http://norvig.com/ngrams/>



Channel Model

$$P(x|w) = \begin{cases} \frac{\text{del}[w_{i-1}, w_i]}{\text{count}[w_{i-1} w_i]}, & \text{if deletion} \\ \frac{\text{ins}[w_{i-1}, x_i]}{\text{count}[w_{i-1}]}, & \text{if insertion} \\ \frac{\text{sub}[x_i, w_i]}{\text{count}[w_i]}, & \text{if substitution} \\ \frac{\text{trans}[w_i, w_{i+1}]}{\text{count}[w_i w_{i+1}]}, & \text{if transposition} \end{cases}$$



Smoothing probabilities: Add-1 smoothing

- But if we use the confusion matrix example, unseen errors are impossible!
- They'll make the overall probability 0. That seems too harsh
 - e.g., in Kernighan's chart $q \rightarrow a$ and $a \rightarrow q$ are both 0, even though they're adjacent on the keyboard!
- A simple solution is to add 1 to all counts and then if there is a $|A|$ character alphabet, to normalize appropriately:

$$\text{If substitution, } P(x | w) = \frac{\text{sub}[x, w] + 1}{\text{count}[w] + A}$$

Noisy channel probability for across

Candidate Correction	Correct Letter	Error Letter	$x w$	$P(x \text{word})$	$P(\text{word})$	$10^9 * P(x w)P(w)$
actress	t	-	c ct	.000117	.0000231	2.7
cress	-	a	a #	.00000144	.000000544	.00078
caress	ca	ac	ac ca	.00000164	.00000170	.0028
access	c	r	r c	.000000209	.0000916	.019
across	o	e	e o	.0000093	.000299	2.8
acres	-	s	es e	.0000321	.0000318	1.0
acres	-	s	ss s	.0000342	.0000318	1.0



Context-sensitive spelling correction

- Determining whether **actress** or **across** is appropriate will require looking at the context of use
- We can do this with a better **language model**
- A **bigram language model** conditions the probability of a word on (just) the previous word

$$P(w_1 \dots w_n) = P(w_1)P(w_2 | w_1) \dots P(w_n | w_{n-1})$$

Incorporating context words

- For unigram counts, $P(w)$ is always non-zero
 - if our dictionary is derived from the document collection
- This won't be true of $P(w_k | w_{k-1})$. We need to **smooth**
- We could use add-1 smoothing on this conditional distribution
- But here's a better way – interpolate a unigram and a bigram:

$$P_{li}(w_k | w_{k-1}) = \lambda P_{uni}(w_k) + (1-\lambda) P_{bi}(w_k | w_{k-1})$$

- $P_{bi}(w_k | w_{k-1}) = C(w_{k-1}, w_k) / C(w_{k-1})$

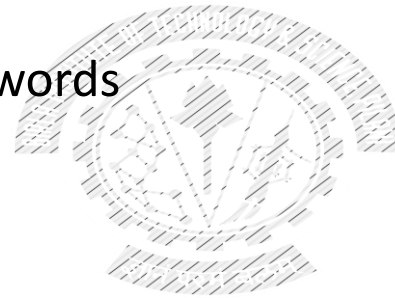
Using a bigram language model

- "a stellar and versatile **acress** whose combination of sass and glamour..."
- Counts from the Corpus of Contemporary American English with add-1 smoothing
- $P(\text{actress}|\text{versatile}) = .000021$ $P(\text{whose}|\text{actress}) = .0010$
- $P(\text{across}|\text{versatile}) = .000021$ $P(\text{whose}|\text{across}) = .000006$
- $P(\text{"versatile actress whose"}) = .000021 * .0010 = 210 \times 10^{-10}$
- $P(\text{"versatile across whose"}) = .000021 * .000006 = 1 \times 10^{-10}$



Solving real-word spelling errors

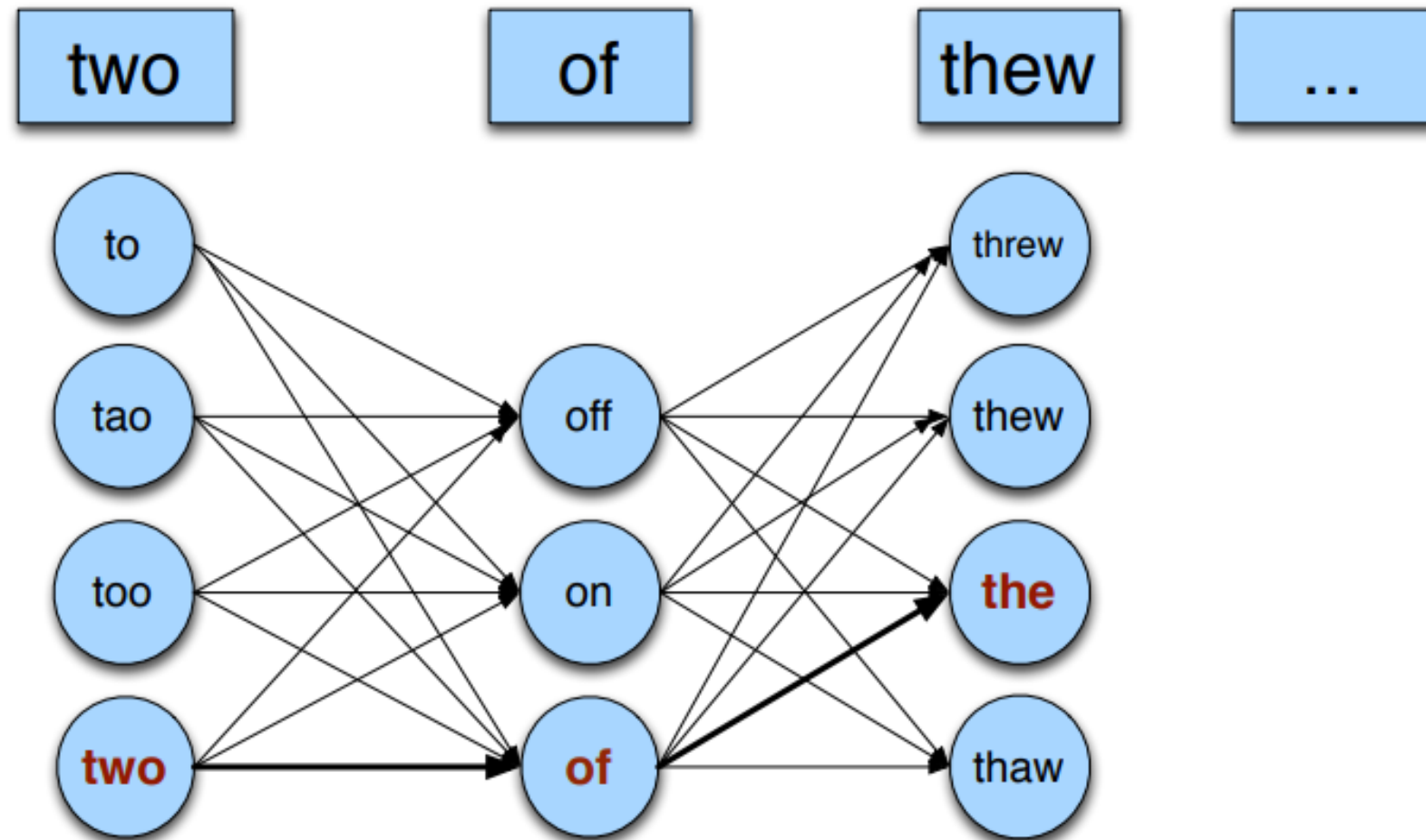
- For each word in sentence
 - Generate candidate set
 - the word itself
 - all single-letter edits that are English words
 - words that are homophones
- Choose best candidates
 - Noisy channel model
 - Task-specific classifier



Noisy channel for real-word spell correction

- Given a sentence $w_1, w_2, w_3, \dots, w_n$
- Generate a set of candidates for each word w_i
 - $\text{Candidate}(w_1) = \{w_1, w'_1, w''_1, w'''_1, \dots\}$
 - $\text{Candidate}(w_2) = \{w_2, w'_2, w''_2, w'''_2, \dots\}$
 - $\text{Candidate}(w_n) = \{w_n, w'_n, w''_n, w'''_n, \dots\}$
- Choose the sequence W that maximizes $P(W)$

Noisy channel for real-word spell correction



Simplification: One error per sentence



- Out of all possible sentences with one word replaced
 - w_1, w''_2, w_3, w_4 two **off** thew
 - w_1, w_2, w'_3, w_4 two of **the**
 - w'''_1, w_2, w_3, w_4 **too** of thew
 - ...
- Choose the sequence W that maximizes $P(W)$

Peter Norvig's "thew" example

x	w	x w	$P(x w)$	$P(w)$	$10^9 P(x w)P(w)$
thew	the	ew e	0.000007	0.02	144
thew	thew		0.95	0.00000009	90
thew	thaw	e a	0.001	0.00000007	0.7
thew	threw	h hr	0.000008	0.0000004	0.03
thew	thwe	ew we	0.000003	0.000000004	0.0001

Machine Learning Approaches



RNNs & LSTMs

Sequence models that capture long-range dependencies. Trained on character/word sequences to predict corrections.

- *char-level seq2seq for spelling*

Transformer Models

BERT, GPT, T5 — pre-trained on massive corpora. Fine-tuned for spell/grammar correction with attention mechanisms.

- *BERT: masked language modeling*

Seq2Seq with Attention

Encoder-decoder architecture maps erroneous sentence to corrected output, learning alignment between error and correction.

- *End-to-end grammatical error correction*

CRF & SVM Classifiers

Conditional Random Fields for token-level tagging. SVMs for binary classification of correct vs. incorrect word usage.

- *Feature-based token classification*

Pre-trained LLMs

GPT-4, Claude, LLaMA — few-shot or zero-shot correction. Can understand context, style, and domain-specific language.

- *In-context learning for correction*

Ensemble Methods

Combine multiple models (rule-based + statistical + neural). Voting or stacking improves precision and recall.

- *Best accuracy in practice*

Open source Spellcheck tools

- **pyspellchecker**: This is a pure Python library and a direct implementation of Peter Norvig's famous statistical spell-checking algorithm. It's simple, easy to use, and self-contained.
- **textblob**: This popular NLP library includes a statistical spellchecker (also based on Norvig's algorithm) as one of its many features.
- **Symspellpy**: This is a high-speed Python library for spelling correction and fuzzy searching based on the Symmetric Delete algorithm.
- **LanguageTool**: Powerful rule-based spellchecker.
- combines these rules with statistical (n-gram) models to catch errors that simple dictionary lookups would miss.
- **Hunspell**: Famous and powerful open-source rule-based spellcheck engine.
It is the spellchecking engine used by Google Chrome, Mozilla Firefox, LibreOffice & OpenOffice, macOS (for many applications)



Open source spellcheck tools

Tool	Type	Primary Use	Context-Aware?
LanguageTool	Hybrid (Rules + ML)	Full Grammar & Style API	Yes (statistical)
Hunspell	Engine / Library	Fast, accurate spellchecking	No
SymSpell	Library	Extreme speed (e.g., search)	No
pyspellchecker	Library (Statistical)	Simple Python spellcheck	No



Open source Spellcheck online tools

- <https://languagetool.org/>
- <https://spellcheck24.net/>
- <https://www.reverso.net/text-translation>
- <https://www.scribens.com/>



DL/ Transformer based Open Source Spell checker Tools

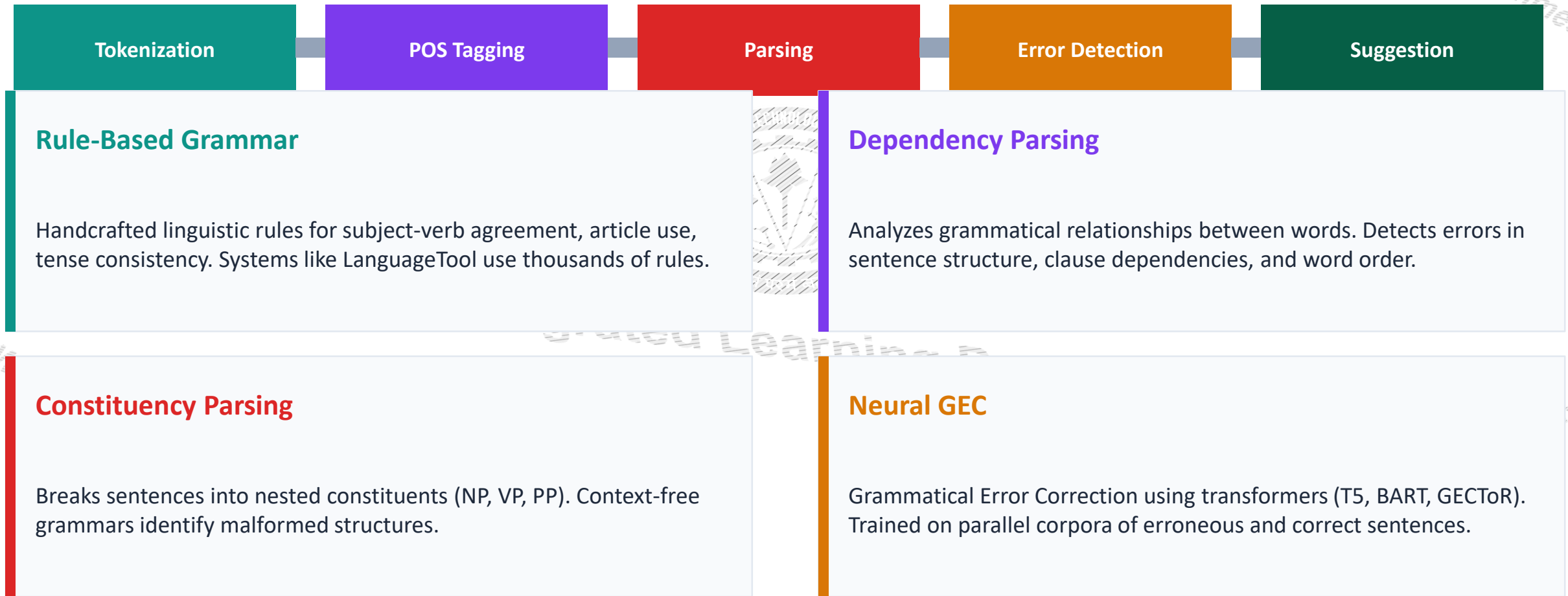


- Pre-trained Models
 - Hugging Face Transformers like T5 that has been fine-tuned on the task of "translating" incorrect text into correct text.
- LanguageTool
 - uses a **hybrid approach**, which is what makes it so robust.
 - **Rule-Based:** A massive set of hand-crafted rules for catching common grammar and spelling errors.
 - **Machine Learning:** Statistical (n-gram) models to find contextual errors (like "I went **two** the store") that the rules miss.
 - **DL/ Transformer based** (used in premium version)
- NeuSpell
 - Implementations of different DL models (BERT-based, LSTM-based, and Seq2Seq models) all pre-trained for spellchecking.

Grammar Checking Techniques



NLP Processing Pipeline:



Transformer-Based Models — Deep Dive



Key Models & Their Roles

BERT	Task: Masked LM fine-tuning ⚡ Bidirectional context	Use case: Real-word error detection
T5	Task: Text-to-text generation ⚡ Encoder-decoder	Use case: GEC, paraphrasing
GPT-4	Task: Causal LM, prompting ⚡ Few-shot correction	Use case: Context-aware suggestion
GECToR	Task: Token-level tagging ⚡ Fast inference	Use case: Production GEC systems

Commercial Grammar Check tools

Tool	Primary Approach	How It Works
Grammarly	Agentic AI & Generative AI	Deploys a team of "AI agents" (e.g., Proofreader, Paraphraser, Tone Adjuster) that proactively analyze context, intent, and style to offer holistic writing suggestions.
QuillBot	Generative AI (LLM)	Its core function is paraphrasing, which is powered by a generative LLM. Its grammar check and other tools are also built with this AI-first approach to rewrite and correct text.
ProWritingAid	Hybrid (Analytical AI + Generative AI)	Uses AI to run 25+ distinct analytical reports (e.g., "Sticky Sentences," "Pacing"). It <i>separately</i> uses generative AI for specific tasks like "Sparks" (rephrasing) and "Critiques."

Open Source Grammar Check tools

Language Tool	Hybrid (Rule-Based + AI)	The free, open-source core is a powerful rule-based engine (using XML/Java rules). The premium version adds a proprietary AI layer for advanced style and rephrasing.
After the Deadline	Statistical NLP (N-grams)	A classic NLP model that uses statistical analysis (n-grams) and neural networks to find "real word errors" by checking the probability of word sequences against a large corpus.
Gramformer	Machine Learning (Transformers)	A Python framework (not a full app) built on Transformer models (like those in LLMs). It's an ML model fine-tuned specifically to detect and correct grammatical errors.

Notable Tools and Models



- **GECToR**: Transformer-based token-level corrector (very fast and accurate)
- **Errant**: Tool for evaluating GEC systems
- **OpenNMT / FairSeq**: Frameworks for building GEC models

System/Model	Approach	Strengths	Weaknesses	Used By
GECToR	Token-level Transformer (edit tagging)	Fast, interpretable, state-of-the-art for GEC	Needs annotated data, limited fluency handling	Open-source, academic
T5 / BART	Seq2Seq transformer (text-to-text)	Powerful fluency correction, few-shot learning	Slow, may hallucinate corrections	General-purpose (used in some GEC research)
BERT / RoBERTa Classifiers	Error detection, often paired with correctors	High accuracy detection	Not standalone for full correction	Grammarly (likely part of stack), research
mT5 / XLM-R	Multilingual Transformers	Supports many languages	Computationally heavy	Google, multilingual GEC tasks
Rule-Based Systems	Hand-crafted grammar and style rules	Transparent, fast	Rigid, low coverage	LanguageTool, old Microsoft Word
Grammarly	Hybrid: deep learning + rules + ensembles	Fluent, context-aware, real-world tool	Proprietary, limited transparency	Grammarly
Google Docs Grammar Tool	Transformer-based, real-time	Good latency, seamless user experience	Mostly grammar, not style	Google Workspace

Agentic AI and Spellcheck/Grammarcheck

- **Preprocessing Step** – Agentic AI often includes a spellcheck and grammar-correction module to clean user input before analysis.
- **Improved Understanding** – Corrected text ensures better intent recognition and semantic parsing for downstream reasoning.
- **Embedded or External Tools** – Systems may use internal NLP models, APIs (like Grammarly or LanguageTool), or LLM-based self-correction.
- **Pipeline Integration** – Grammar/spell check sits in the preprocessing stage, feeding into semantic analysis → reasoning → action/output loop.

Evaluation & Challenges



Evaluation Metrics

Precision: Correct suggestions / Total suggestions made

Recall: Errors caught / Total actual errors

F0.5 Score: Weighted F-measure favoring precision (GEC standard)

ERRANT: Error type analysis — classifies correction categories

BLEU / ROUGE: Text overlap metrics for generated corrections

Benchmarks: CoNLL-2014, BEA-2019, JFLEG

Key Challenges

- Context sensitivity: 'their/there/they're'
- Domain adaptation: medical, legal jargon
- Multilingual support & code-switching
- Preserving author intent and style
- Low-resource languages lack training data
- Real-time performance requirements
- Handling informal/social media text

Precision

How accurate are our suggestions?

innovate

achieve

lead

DEFINITION

Of all the corrections the system made, Precision measures what fraction were actually correct. It penalizes false positives — wrong suggestions.

Why it matters: In GEC, high precision means fewer wrong "corrections" that break the original text. F0.5 score weights precision more heavily than recall.

EXAMPLE

System flags 10 words as errors and suggests corrections.

→ 8 corrections are genuinely correct

→ 2 are wrong (false positives)

$$\text{Precision} = 8 / (8 + 2) = 0.80$$

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

TP = True Positives

FP = False Positives

Interpretation

1.0 Every suggestion was correct

0.5 Half of suggestions were correct

0.0 No suggestion was correct

GEC standard: F0.5 rewards precision over recall

Recall

How many real errors did we catch?

innovate

achieve

lead

DEFINITION

Of all the actual errors in the text, Recall measures what fraction the system successfully detected and corrected. It penalizes misses.

Why it matters: High recall ensures errors aren't missed. However, maximising recall alone can flood text with unnecessary corrections. Balance with Precision.

EXAMPLE

A document has 20 actual spelling/grammar errors.

→ System catches 14 of them

→ 6 errors are missed (false negatives)

$$\text{Recall} = 14 / (14 + 6) = 0.70$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

TP = True Positives

FN = False Negatives

Interpretation

1.0 All errors were caught

0.5 Half the errors were missed

0.0 No errors were detected

Low recall = many errors slip through undetected

F0.5 Score Weighted harmonic mean — GEC standard metric



DEFINITION

F0.5 combines Precision and Recall into a single score, but gives Precision twice the weight. This is the standard metric for Grammatical Error Correction.

Why it matters: Wrong suggestions are more harmful than missed errors — they introduce new mistakes. F0.5 captures this asymmetry, making it the go-to metric in CoNLL-2014, BEA-2019.

EXAMPLE

Given: Precision = 0.80, Recall = 0.70

$$\begin{aligned} F0.5 &= (1.25 \times 0.80 \times 0.70) / (0.25 \times 0.80 + 0.70) \\ &= 0.70 / 0.90 \approx 0.778 \end{aligned}$$

Note: $F0.5 > F1$ when $P > R$ ✓

$$F0.5 = \frac{(1 + \beta^2) \times P \times R}{\beta^2 \times P + R}$$

where $\beta = 0.5$

Interpretation

$\beta = 0.5$ Precision weighted 2× over Recall

$\beta = 1.0$ Standard F1 — equal P & R weight

$\beta = 2.0$ F2 — Recall weighted more

Standard at CoNLL-2014, BEA-2019, JFLEG benchmarks

DEFINITION

ERRANT automatically classifies correction edits into linguistic error types (spelling, morphology, syntax...). Enables per-category F0.5 analysis beyond aggregate scores.

Why it matters: Aggregate F0.5 hides which error types a model handles well vs poorly. ERRANT reveals if a system excels at spelling but fails at grammar — crucial for targeted improvement.

EXAMPLE

Input: "She go to school yesterday."

ERRANT tags:

→ "go" → "went" : VERB:TENSE

→ (no other edits)

Enables per-type precision/recall breakdown

Edit → Error Type

SPELL: 'recieve' → 'receive'

MORPH: 'runned' → 'ran'

ORTH: 'iphone' → 'iPhone'

PREP: 'on' → 'in'

Interpretation

SPELL Spelling & token-level errors

MORPH Morphology / verb form errors

SYNTAX Word order & structural issues

Used in BEA-2019 shared task for fine-grained scoring

NLP Evaluation Metrics

- True Positive (TP) “hit”
 - Flags *I am going for walk this afternoon.*
- False Positive (FP)
 - Flags *I am going for a walk this afternoon.*
- True Negative (TN)
 - Not flag *I am going for a walk this afternoon.*
- False Negative (FN) “miss”
 - Not flag *I am going for walk this afternoon.*



NLP Evaluation Metrics

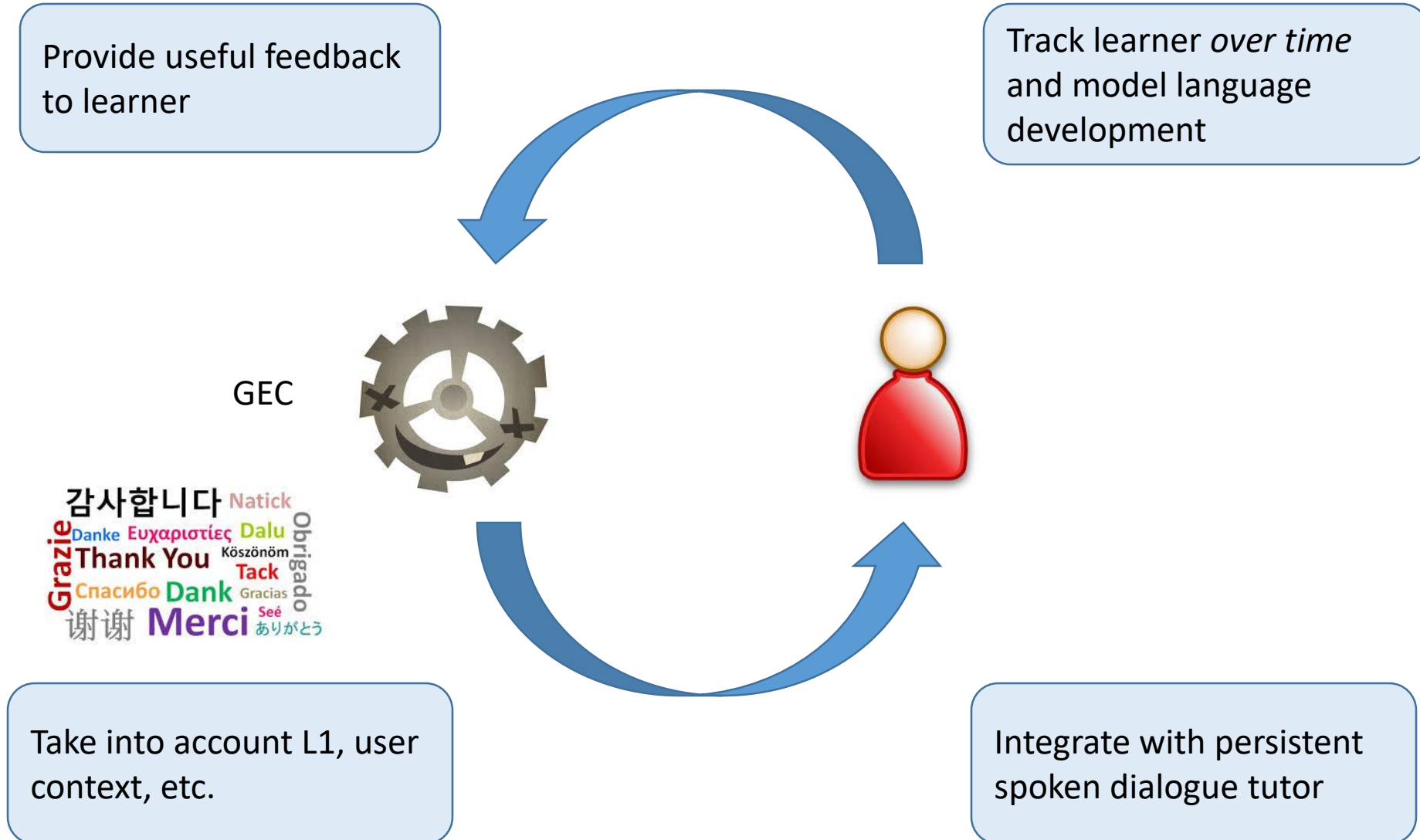
$$\textit{Precision} = \frac{TPs}{TPs + FPs}$$

$$\textit{Recall} = \frac{TPs}{TPs + FNs}$$

$$F - \textit{score} = \frac{\textit{Precision} * \textit{Recall}}{\textit{Precision} + \textit{Recall}}$$

$$\textit{Accuracy} = \frac{TPs + TNs}{TPs + TNs + FPs + FNs}$$

What is the future of GEC?



What is the future of GEC?

- A system which can automatically transform one noisy sentence to a fluent sentence...without a change in meaning

Having discuss all this I must say that I must rather prefer to be a leader than just a member.



After discussing all this I must say that I'd prefer to be a leader than a follower.

What is the future of GEC?

- System need not simply be a text to text transformation, could also take into account:
 - Other sentences in document
 - Context of document (writer's intention)
 - Register
 - Who the document is for
 - Prior sentences writer has produced (personalization)

Scope of research in Spellcheck and Grammmarcheck

No.	Research Area	Focus	Example / Challenge
1	Context-aware models	Semantic understanding	“Their” vs “there”
2	Multilingual & code-mixed text	Cross-language adaptation	Hindi-English text
3	Privacy & on-device NLP	Local model deployment	Federated learning
4	User/domain adaptation	Personalized writing feedback	Legal or academic writing
5	Evaluation & explainability	Human-centered metrics	Transparent suggestions

Recent Research Trends



- **Multilingual grammar correction**
- **Low-resource grammar correction** using synthetic error generation
- **Unsupervised and semi-supervised GEC**
- **Reinforcement Learning** for fluency-aware correction
- **Explainable GEC models** (interpretable edits)

References



- Speech and Language processing: An introduction to Natural Language Processing, Computational Linguistics and speech Recognition by Daniel Jurafsky and James H. Martin[3rd edition] – Appendix B
- Dan Jurafsky Course :
 - https://www.youtube.com/watch?v=dAf_vcs7DuE
- Pawan Goyal Course:
 - <https://www.youtube.com/watch?v=HmcolVdXVpE>
 - <https://sanketp.medium.com/language-models-spellchecking-and-autocorrection-dd10f739443c>
 - <https://www.youtube.com/watch?v=4htP-hmIBvc>
 - <https://nlp.stanford.edu/IR-book/html/htmledition/spelling-correction-1.html>
 - <https://www.geeksforgeeks.org/spelling-checker-in-python/>
 - <https://www.slideshare.net/AmirShokri2/personalized-spell-checking-using-neural-networks>
 - https://www.youtube.com/watch?v=2RU5egfeb_g

References



- https://nlp.johnsnowlabs.com/analyze_spelling_grammar/
- <https://direct.mit.edu/coli/article/49/3/643/115846/Grammatical-Error-Correction-A-Survey-of-the-State>
- <https://blogs.nvidia.com/blog/microsoft-triton-ai-grammar-word/>
- <https://paperswithcode.com/datasets?q=&v=lst&o=newest&task=grammatical-error-correction&mod=texts&page=1>
- <https://onlinelibrary.wiley.com/doi/10.1002/spe.2110>
- <https://www.youtube.com/watch?v=3rVn14m8zaM>
- <https://www.youtube.com/watch?v=pYV8OydsnQA>
- <https://www.youtube.com/watch?v=9ZkM-jn-EWA>
- <https://www.hindawi.com/journals/sp/2021/4213791/>
- <https://www.youtube.com/watch?v=3rVn14m8zaM>
- <https://sapling.ai/grammar-check>
- https://www.youtube.com/watch?v=2RU5eqfeb_g
- <https://www.youtube.com/watch?v=0MiFUES-0F4>
- [Analyze Spelling & Grammar - Spark NLP Demos & Notebooks](#)
- <https://colab.research.google.com/github/piyush0511/SpellChecker-AutoCorrect/blob/main/SpellCheck%20-%20seq2seq.ipynb#scrollTo=eqcJXSpf0fPW>
- https://colab.research.google.com/drive/1ysEKrw_LE2jMndo1snrZUh5w87LQsCxx